

Question 1: Heuristic design and admissibility

Steps

Equation: $h(n) = \text{number of mismatched positions}$

For steps, I look at how many letters are different between the current word and the goal word. Since I have to change one letter at a time, I cannot fix multiple wrong letters all at once. So if three letters are different, I know I need at least three changes to reach the goal.

Because of that, this cannot overestimate the real cost. I might have to make extra changes along the way, but I cannot do it in fewer changes than the number of letters that are different.

Scrabble

Equation: $h(n) = \sum \text{Scrabble value of each goal letter that does not match}$

With Scrabble, I still look at the letters that are different, but instead of just counting them, I use the Scrabble cost of the letters I still need to change them into. The reason is that if a letter in the goal is not there yet, then at some point I have to change that position into the goal letter and pay its Scrabble cost.

It cannot overestimate because I am only adding the costs for letters that I know I still have to put into the word. If I have to make other changes before reaching the goal, then the real cost would only become larger.

Frequency

Equation: $h(n) = \text{number of mismatched letters} + \text{rarity}(\text{goal})$

For frequency, every move costs: $1 + \text{rarity of the new word}$. So I use the number of letters that are different, since I know I need at least that many changes, and I also include the rarity of the goal word because at some point I have to enter the goal word. If I am already at the goal, then the heuristic is 0.

This cannot overestimate because I am only counting costs that I know have to happen. I am not including the rarity costs of all the other words I might have to go through, so the actual path could still cost more.

Question 2: Does the heuristic actually help?

Word pair	Cost measure	UCS costs	UCS nodes expanded	A* cost	A* nodes expanded	reduction
why -> Ask	Steps	5	773	5	16	97.93%
why -> Ask	scrabble	9	765	9	13	98.30%
why -> Ask	frequency	38.855	808	38.855	634	21.53%
bear -> duck	steps	4	691	4	6	99.13%
bear -> duck	scrabble	11	2981	11	6	99.80%
bear -> duck	frequency	38.240	1084	38.240	302	72.14%

Why -> Ask

$$100((773-16)/773) = 97.93\%$$

bear -> duck

$$100((691-6)/691) = 99.13\%$$

That means A* expanded about **99.13% fewer nodes** than UCS.

1. Yes. UCS and A* returned the same optimal cost for both WHY → ASK and BEAR → DUCK under all three cost measures. There was no difference in the final solution cost between the two search methods.

2. For WHY → ASK, A* had the largest percentage reduction under the Scrabble cost, with about 98.30% fewer expanded nodes.

For BEAR → DUCK, A* also had the largest percentage reduction under the Scrabble cost, with about 99.80% fewer expanded nodes.

3. both WHY → ASK and BEAR → DUCK, the frequency heuristic was the least informative. It had the smallest percentage reduction in expanded nodes compared to the steps and Scrabble heuristics.

4. The steps heuristic is helpful because it counts how many letters still need to change. The Scrabble heuristic is even more informative because it also uses the value of those letters. The frequency heuristic is less informative because the cost depends on how rare the words are, which is harder to predict just from how close the word looks to the goal.

Question 3: Reversibility and direction-dependent costs

steps

Yes. For steps, the optimal cost from W1 to W2 will be the same as from W2 to W1. Every legal change costs 1, so reversing the same path gives the same total number of steps.

Scrabble

No. The action can be reversed, but the cost depends on the letter I am changing into. For example, BEAR $\rightarrow$ DEAR changes B to D, which costs 2, while DEAR $\rightarrow$ BEAR changes D to B, which costs 3. More generally, when a whole path is reversed, the difference in cost depends only on the Scrabble values of the starting and ending words, so the optimal costs do not have to be equal.

Frequency

No. The action can be reversed, but the cost depends on the rarity of the word I am entering. For a path from W1 to W2, reversing the path switches which endpoint's rarity is paid. So if W1 and W2 have different rarity values, the optimal costs in the two directions will also be different.

So, reversibility means I can make the change in both directions, but it does not mean both directions have the same cost.

Question 4: LLM heuristic audit

1. I used Gemini 3.6 Flash with this prompt “In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost. “ $h_1(n)=h_0(n)(1+r_{\min})$ ”

Precise Definition of Proposed Heuristic $h_1(n)$

Let:

- $w = n$ be the current word (state) of length $L \in \{3, 4\}$.
- g be the goal word.
- D be the dictionary of valid words, where each word $v \in D$ has an associated non-negative cost value $\text{rarity}(v) \geq 0$.
- $H(n, g) = \{i \in \{1, \dots, L\} \mid n[i] \neq g[i]\}$ be the set of mismatched letter indices between state n and goal g .
- $h_0(n) = |H(n, g)|$ be the baseline Hamming distance heuristic.
- $r_{\min} = \min_{v \in D} \text{rarity}(v) \geq 0$ be the minimum rarity across all words in the dictionary.

We define the proposed rarity-aware heuristic $h_1(n)$ as:

$$h_1(n) = h_0(n) + h_0(n) \cdot r_{\min} = h_0(n) \cdot (1 + r_{\min})$$

2. Precise formula given was $h_1(n) = h_0(n)(1 + r_{\min})$
3. I think heuristic is admissible because I still have to change at least the number of letters that are different from the goal. Also, every change costs at least $1 + r_{\min}$. Because of that, this heuristic shouldn't be larger than the real remaining cost.
4. I noticed one problem with Gemini's heuristic. If the smallest rarity value is 0, then the new heuristic becomes the same as the original heuristic, so it would not actually be more informative. A simple fix is to also include the rarity of the goal word, since I know I eventually have to enter the goal word anyway.
5. I tested the repaired heuristic on both word pairs. It gave the same solution costs and expanded the same number of nodes as the original heuristic. So for these tests, the repaired heuristic did not improve A*.

```
why -> ask
634 paths have been expanded and 213 paths remain in the frontier
Solution cost: 38.85540652

bear -> duck
302 paths have been expanded and 806 paths remain in the frontier
Solution cost: 38.2402284
[milany1@linux1 PA1]$
```